

Трекер личных расходов

Облачные вычисления - Этапы 1,2 и 3

Платформа: Yandex Cloud

1. Этап 1: Выбор реализуемой идеи

1.1. Описание проблемы

Студенты и молодые люди часто не понимают, куда уходят деньги: расходы совершаются импульсивно, систематический учёт не ведётся, и в конце месяца приходит неприятное осознание того, что бюджет незаметно ушёл на совершенно незначительные вещи.

Существующие мобильные приложения либо перегружены функциональностью, либо требуют привязки банковской карты, что вызывает обоснованные опасения относительно конфиденциальности данных. Встроенные трекеры в банковских приложениях автоматически категоризируют расходы, однако делают это неточно: покупка в супермаркете может попасть в категорию «развлечения», а оплата такси - в «переводы». Кроме того, при наличии карт нескольких банков единой картины расходов получить не удаётся.

1.2. Предлагаемое решение

RESTful-сервис для ручного учёта личных расходов. Пользователь самостоятельно вносит записи о тратах, указывая сумму, категорию и комментарий. Ручной ввод является осознанным выбором: он заставляет пользователя замечать каждую трату, а не полагаться на автоматическую категоризацию. Сервис хранит историю расходов и предоставляет возможность просматривать, редактировать и удалять записи через API.

Границы MVP. Данная реализация является однопользовательским MVP - регистрация и авторизация намеренно исключены из области видимости, чтобы сосредоточиться на облачной инфраструктуре. Все операции выполняются от имени единственного пользователя системы. При необходимости расширения до многопользовательского режима достаточно добавить таблицу `users` и поле `user_id` в таблицу расходов.

1.3. Потенциальные компоненты

Для работы сервиса необходима **виртуальная машина** - она обеспечивает постоянно доступный HTTP-сервер, на котором будет запущено приложение в Docker-контейнере. Это также является основой для дальнейшего развёртывания в Kubernetes на последующих этапах работы.

Для хранения записей о расходах необходима **реляционная база данных** - данные структурированы и однородны (сумма, категория, дата), а SQL позволяет удобно фильтровать и агрегировать их, например подсчитывать сумму расходов по категориям за выбранный период. База данных PostgreSQL будет развёрнута в Docker-контейнере на той же виртуальной машине.

В качестве третьего компонента необходим **API Gateway** - он обеспечивает единую точку входа для HTTP-запросов и маршрутизирует их к приложению, что позволяет реализовать полноценный REST-интерфейс.

Для последующих этапов работы (4.2-4.4) также потребуется **Container Registry** для хранения Docker-образов. Оркестрация контейнеров на этапе 4.3 реализуется через **single-node Kubernetes** (minikube или k3s), развёрнутый на той же виртуальной машине. Использование Managed Service for Kubernetes не предусмотрено бюджетом проекта (задание не устанавливает требования к типу Kubernetes-кластера).

1.4. Функциональные требования - 4 CRUD-операции

- **Create** - добавить новую запись о расходе (сумма, категория, дата траты, комментарий).
- **Read** - получить список расходов с возможностью фильтрации по дате и категории.
- **Update** - изменить сумму, категорию или комментарий существующей записи.
- **Delete** - удалить запись о расходе.

1.5. Схема базы данных

Таблица **expenses**. Реализация является однопользовательской, поэтому поле **user_id** отсутствует намеренно.

Поле	Тип	Описание
id	UUID	Уникальный идентификатор записи
amount	NUMERIC(10,2)	Сумма расхода в рублях
category	VARCHAR(64)	Категория (еда, транспорт, развлечения и др.)
note	TEXT	Произвольный комментарий пользователя
expense_date	DATE	Фактическая дата траты (может отличаться от даты создания записи)
created_at	TIMESTAMP	Дата и время создания записи в системе
updated_at	TIMESTAMP	Дата и время последнего изменения записи

2. Этап 2: Выбор платформы и стека

2.1. Выбор платформы

В качестве целевой платформы выбран **Yandex Cloud**. При выборе рассматривались три провайдера:

Провайдер	Грант	Срок	Примечание
Yandex Cloud	4 000 руб.	60 дней	Выбран
VK Cloud	3 000 руб.	60 дней	Меньший грант
Timeweb Cloud	3 000 руб.	60 дней	Меньший грант

Yandex Cloud выбран по совокупности факторов: наибольший грант (4000 руб.), полная русскоязычная документация, а также наличие всех необходимых сервисов - Compute Cloud, Container Registry и API Gateway - в рамках единой платформы. Компоненты одного провайдера работают в одной виртуальной сети, что упрощает настройку доступа между ними. Kubernetes разворачивается как single-node кластер (minikube/k3s) на виртуальной машине.

2.2. Выбранные облачные сервисы

2.2.1. Yandex Compute Cloud (виртуальная машина)

Виртуальная машина является основой инфраструктуры: на ней в Docker-контейнерах запускается RESTful-сервис на FastAPI и база данных PostgreSQL. Такой подход выбран намеренно - он обеспечивает прямую совместимость с этапами 4.2-4.3, где требуется собрать Docker-образ, сохранить его в Container Registry и развернуть в Kubernetes.

Выбор конфигурации. Рассматривались конфигурации с различными долями гарантированного vCPU. Тип с долей 5% оказался недостаточным для стабильной работы Docker. Выбрана конфигурация с **20% гарантированной долей vCPU** - она обеспечивает предсказуемую производительность и при этом значительно дешевле полной гарантии 100%.

Итоговая конфигурация: платформа **Intel Broadwell**, гарантированная доля **20% vCPU**, **2 vCPU**, **6 ГБ RAM**, хранилище **network-hdd 60 ГБ**. Стоимость: **3 283,61 руб./мес.**

Были выбраны именно 6 ГБ RAM и 60 ГБ диск. Анализ требований этапов 4.2-4.3 показал, что конфигурация 4 ГБ RAM и 5 ГБ диска недостаточна: minikube требует минимум 2 ГБ свободной памяти, а Docker-образы приложения, PostgreSQL, Prometheus и Grafana в сумме занимают более 5 ГБ на диске. Выбранная конфигурация обеспечивает комфортный запас для развёртывания полного стека включая мониторинг.

2.2.2. PostgreSQL в Docker-контейнере

База данных PostgreSQL разворачивается в Docker-контейнере на той же виртуальной машине. Данные расходов строго структурированы, ключевые операции - фильтрация по дате, группировка по категории, подсчёт суммы - хорошо выражаются через SQL.

Managed PostgreSQL не была выбрана из-за стоимости. Минимальная конфигурация Managed Service for PostgreSQL стоит от 3 189 руб./мес, что в сумме с виртуальной машиной превысило бы грант.

2.2.3. Yandex Container Registry

Облачный реестр для хранения Docker-образов приложения. Используется начиная с этапа 4.2: после сборки образ публикуется в Container Registry, откуда его забирает Kubernetes при развёртывании. Бесплатный порог - **1 ГБ** хранилища в месяц.

2.2.4. Yandex API Gateway

Единая точка входа для HTTP-запросов. Принимает запросы от клиента и маршрутизирует их к приложению на виртуальной машине. При нагрузке до 100 000 запросов в месяц сервис полностью бесплатен (**0,00 руб./мес**).

2.3. Схема взаимодействия компонентов

Клиент → API Gateway → FastAPI (Docker) → PostgreSQL (Docker)

(FastAPI и PostgreSQL развёрнуты в Docker-контейнерах на Yandex Compute Cloud)

CI/CD → Container Registry → Kubernetes (этап 4.3)

Пользователь отправляет HTTP-запрос, API Gateway маршрутизирует его на виртуальную машину, FastAPI-приложение обрабатывает запрос, выполняет операцию с PostgreSQL и возвращает результат.

CI/CD-пайплайн собирает Docker-образ и публикует его в Container Registry, откуда он впоследствии разворачивается в Kubernetes.

2.4. Оценка стоимости (FinOps)

Расчёт выполнен исходя из предполагаемой нагрузки: один активный пользователь, около 10 запросов в день. Виртуальная машина работает на протяжении всего активного периода разработки (1 месяц) - она обеспечивает постоянную доступность приложения. В соответствии с практиками FinOps тяжёлые вспомогательные компоненты (minikube, Prometheus, Grafana, SonarQube) запускаются исключительно на период демонстрации и сдачи соответствующих этапов, а не держатся поднятыми 24/7. Цены получены из официального калькулятора Yandex Cloud, все суммы указаны с НДС.

Сервис	Конфигурация	Стоимость
Compute Cloud (BM)	Intel Broadwell, 20% vCPU, 2 vCPU, 6ГБ RAM, 60ГБ HDD	3 283,61 руб./мес
PostgreSQL в Docker	на той же BM	0,00 руб.
Container Registry	до 1 ГБ хранилища	0,00 руб.
API Gateway	до 100K запросов/мес	0,00 руб.
Итого за 1 месяц		3 283,61 руб.

Суммарная стоимость **3 283,61 руб.** укладывается в грант **4 000 руб.** с резервом **≈716 руб.**

3. Роли и сценарии использования

3.1. Роли

Система является однопользовательским MVP, поэтому определена одна роль:

- **Пользователь** - единственный актор системы. Имеет полный доступ ко всем CRUD-операциям над записями о расходах. Авторизация намеренно исключена из области видимости MVP.

3.2. Доступная функциональность

Функция	Доступ	Примечание
Добавить запись о расходе	Полный	Обязательны сумма, категория, дата
Просмотреть список расходов	Полный	Фильтрация по категории и периоду
Редактировать запись	Полный	Изменение любого поля
Удалить запись	Полный	Безвозвратное удаление
Фильтрация	Полный	query-параметры
Регистрация / авторизация	Нет	Намеренно исключено из MVP

3.3. Сценарии использования

3.4. Диаграмма вариантов использования

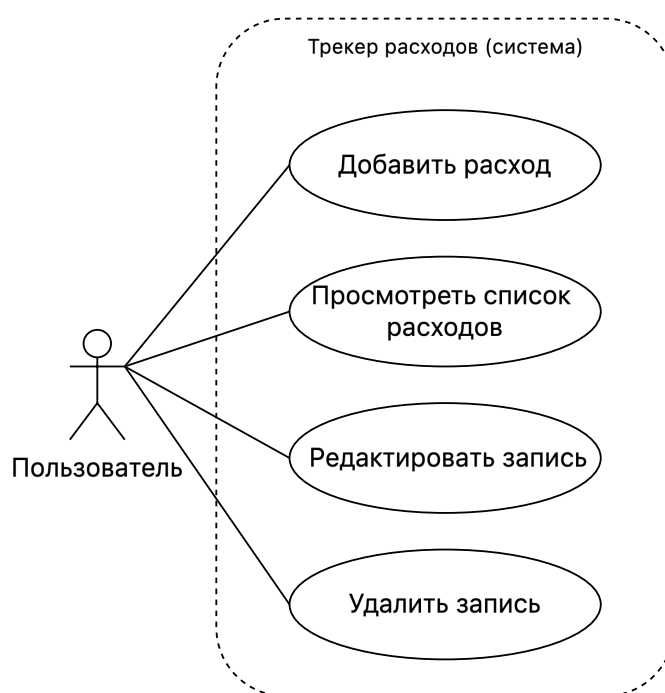


Рис. 1: Диаграмма вариантов использования

3.4.1. Добавление записи о расходе

Участник: Пользователь.

Предусловие: Приложение открыто в браузере.

Основной поток:

1. Пользователь заполняет форму: сумма, категория, дата траты, комментарий.
2. Клиент отправляет POST /expenses.
3. Сервер сохраняет запись в БД и возвращает созданный объект.
4. Клиент отображает новую запись в списке.

Альтернативный поток: сумма отрицательная или не указана - сервер возвращает 422 **Unprocessable Entity**, клиент показывает ошибку валидации.

3.4.2. Просмотр списка расходов

Участник: Пользователь.

Предусловие: Приложение открыто в браузере.

Основной поток:

1. Пользователь открывает главную страницу или применяет фильтры (категория, период дат).
2. Клиент отправляет **GET /expenses** с query-параметрами.
3. Сервер возвращает отфильтрованный список расходов.
4. Клиент отображает список и итоговую сумму за период.

3.4.3. Редактирование записи

Участник: Пользователь.

Предусловие: Приложение открыто в браузере.

Основной поток:

1. Пользователь нажимает «Редактировать» напротив записи.
2. Клиент заполняет форму текущими значениями.
3. Пользователь изменяет нужные поля и подтверждает.
4. Клиент отправляет **PUT /expenses/{id}**.
5. Сервер обновляет запись и возвращает обновлённый объект.

Альтернативный поток: запись с указанным **id** не найдена - сервер возвращает 404 **Not Found**.

3.4.4. Удаление записи

Участник: Пользователь.

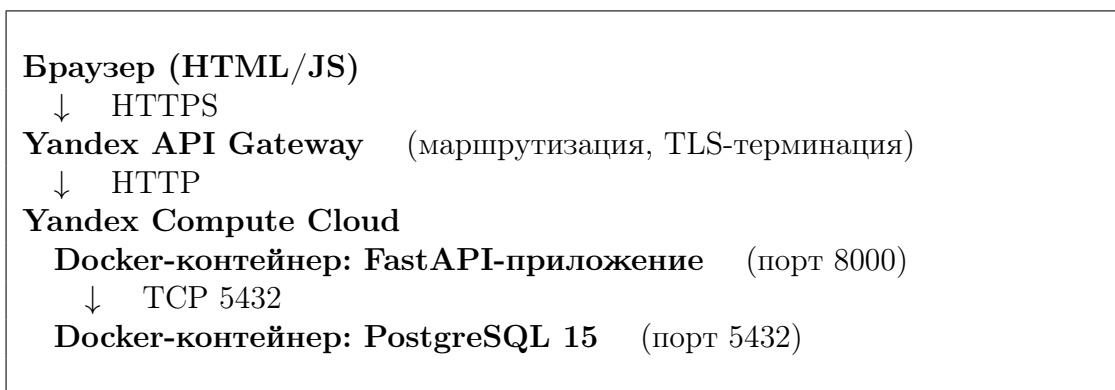
Предусловие: Приложение открыто в браузере.

Основной поток:

1. Пользователь нажимает «Удалить» напротив записи.
2. Клиент отправляет **DELETE /expenses/{id}**.
3. Сервер удаляет запись и возвращает 204 **No Content**.
4. Клиент убирает запись из списка.

4. Архитектурный ландшафт

4.1. Общая схема



4.2. Архитектурная диаграмма

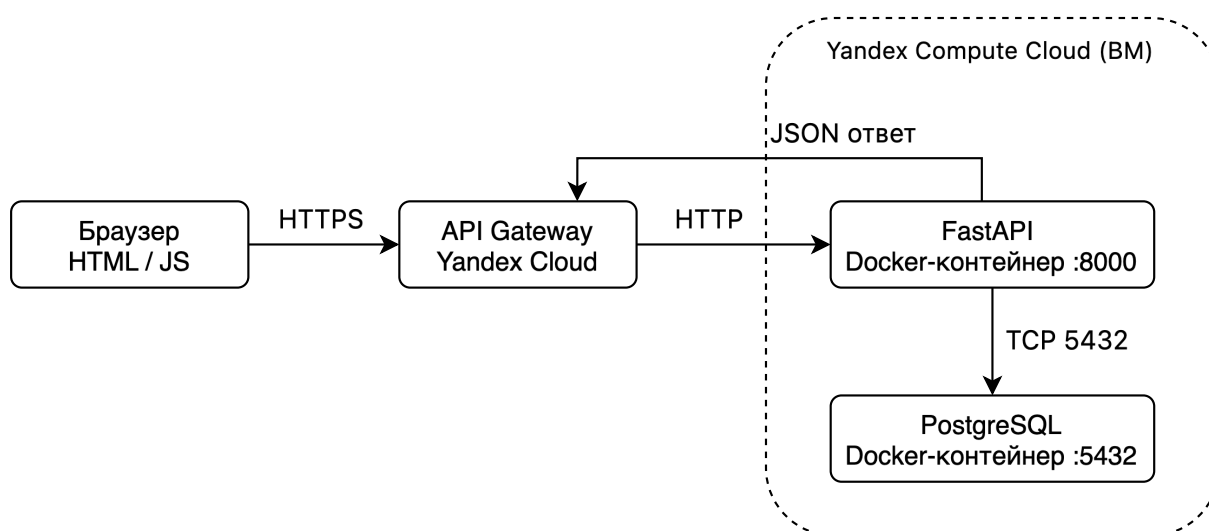


Рис. 2: Архитектурный ландшафт решения

4.3. Компоненты и их ответственность

Компонент	Ответственность
Браузер (HTML/JS)	Отображение списка расходов, форм создания и редактирования.
Yandex API Gateway	Приём HTTPS-запросов от клиента. Маршрутизация на <code>http://{VM_IP}:8000</code> . TLS-терминация.
FastAPI (Docker)	Обработка HTTP-запросов, валидация входных данных, выполнение SQL-запросов через SQLAlchemy, формирование JSON-ответов.
PostgreSQL (Docker)	Персистентное хранение таблицы <code>expenses</code> . Индексирование по <code>expense_date</code> и <code>category</code> .
Yandex Container Registry	Хранение Docker-образа приложения. Используется CI/CD-пайплайном и Kubernetes.
minikube (этап 4.3)	Single-node Kubernetes-кластер на той же ВМ для оркестрации, НРА и мониторинга.

5. Схема базы данных

Таблица `expenses`. Реализация однопользовательская, поэтому поле `user_id` отсутствует намеренно.

Поле	Тип	Описание
<code>id</code>	UUID	Первичный ключ, генерируется автоматически
<code>amount</code>	NUMERIC(10,2)	Сумма расхода
<code>category</code>	VARCHAR(64)	Категория расхода (еда, транспорт и др.)
<code>note</code>	TEXT	Произвольный комментарий, может быть NULL
<code>expense_date</code>	DATE	Фактическая дата траты
<code>created_at</code>	TIMESTAMP	Дата создания записи, DEFAULT now()
<code>updated_at</code>	TIMESTAMP	Дата изменения, обновляется триггером

Индексы:

- `idx_expenses_date` на `expense_date` - ускоряет фильтрацию по периоду.
- `idx_expenses_category` на `category` - ускоряет группировку по категории.

5.1. Хранилище объектов

Объектное хранилище (Yandex Object Storage) в данном решении не используется. Все данные приложения - текстовые записи о расходах - хранятся в реляционной БД PostgreSQL. Объем данных за год составляет менее 1 МБ, что не требует отдельного объектного хранилища.

Расширение: при добавлении функции загрузки чеков или скриншотов к записям расходов целесообразно подключить Yandex Object Storage. В этом случае в таблицу `expenses` добавляется поле `receipt_url` VARCHAR со ссылкой на объект в хранилище.

6. Описание API

Базовый URL: `https://{api-gateway-id}.apigw.yandexcloud.net`

Формат данных: `application/json`.

Все даты передаются в формате YYYY-MM-DD.

6.1. Диаграмма последовательности

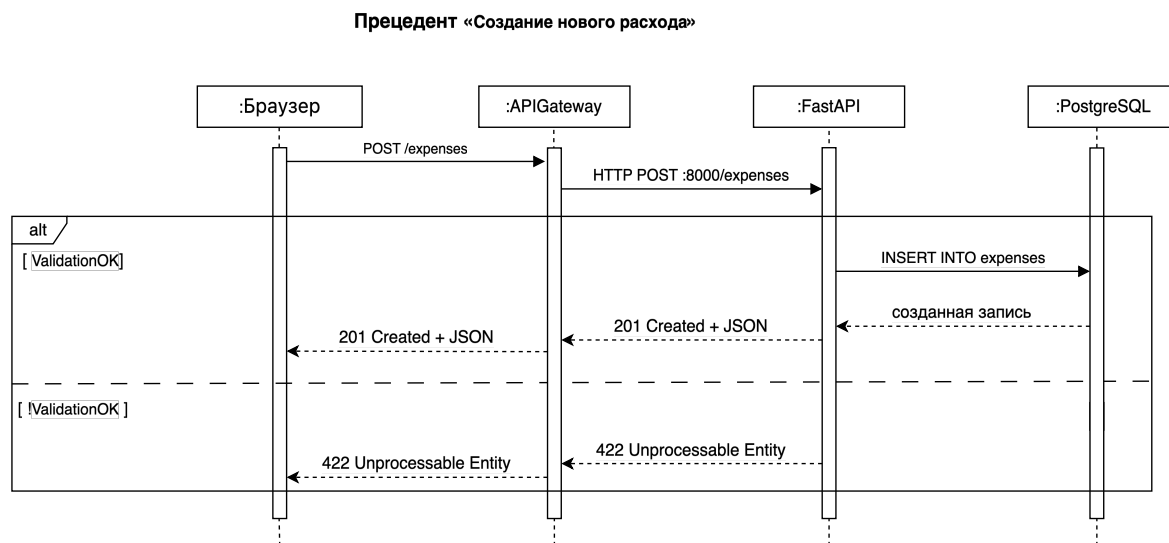


Рис. 3: Диаграмма последовательности: создание расхода (UC-1)

6.2. POST /expenses - создать расход

Тело запроса:

```
{
  "amount": 450.00,
  "category": "food",
  "expense_date": "2025-04-05",
  "note": "lunch"
}
```

Ответ 201 Created:

```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "amount": 450.00,
  "category": "food",
  "expense_date": "2025-04-05",
  "note": "lunch",
  "created_at": "2025-04-05T12:00:00",
  "updated_at": "2025-04-05T12:00:00"
}
```

Ошибки: 422 - ошибка валидации (отрицательная сумма, отсутствует обязательное поле).

6.3. GET /expenses - получить список расходов

Query-параметры (все опциональны):

- category - фильтр по категории.
- date_from, date_to - фильтр по периоду (YYYY-MM-DD).

- **skip** (по умолчанию 0), **limit** (по умолчанию 100) - пагинация.

Ответ 200 OK:

```
{
  "total": 2,
  "total_amount": 950.00,
  "items": [
    { "id": "...", "amount": 450.00, "category": "food", ... },
    { "id": "...", "amount": 500.00, "category": "transport", ... }
  ]
}
```

6.4. PUT /expenses/{id} - обновить расход

Тело запроса (все поля опциональны):

```
{
  "amount": 500.00,
  "category": "transport",
  "note": "correction"
}
```

Ответ 200 OK: обновлённый объект расхода (аналогично POST).

Ошибки: 404 - запись не найдена, 422 - ошибка валидации.

6.5. DELETE /expenses/{id} - удалить расход

Ответ 204 No Content: тело ответа отсутствует.

Ошибки: 404 - запись не найдена.

6.6. Сводная таблица эндпоинтов

Метод	Путь	Успех	Описание
POST	/expenses	201	Создать запись о расходе
GET	/expenses	200	Список расходов с фильтрами
PUT	/expenses/{id}	200	Обновить запись
DELETE	/expenses/{id}	204	Удалить запись
GET	/health	200	Проверка работоспособности сервиса

6.7. Взаимодействие FastAPI и PostgreSQL

FastAPI обращается к PostgreSQL через библиотеку **SQLAlchemy** (асинхронный драйвер **asyncpg**). Соединение устанавливается по протоколу **TCP на порту 5432** внутри Docker. Строка подключения передаётся через переменную окружения:

```
DATABASE_URL=postgresql+asyncpg://
  user:password@db:5432/expenses_db
```

Каждый HTTP-запрос к API открывает сессию SQLAlchemy (AsyncSession), которая берёт соединение из пула подключений к PostgreSQL, выполняет один или

несколько SQL-запросов и возвращает соединение обратно в пул. Размер пула оставлен по умолчанию, что достаточно для однопользовательского MVP.

Операция API	SQL-запрос	Примечание
POST /expenses	INSERT INTO expenses	Возвращает созданную строку
GET /expenses	SELECT ... WHERE ...	Фильтрация + LIMIT/OFFSET
PUT /expenses/id	UPDATE expenses SET	WHERE id = :id
DELETE /expenses/id	DELETE FROM expenses	WHERE id = :id
GET /health	SELECT 1	Проверка доступности БД

6.8. Обработка ошибок

Код	Ситуация	Тело ответа
422	Ошибка валидации Pydantic	{"detail": [{"loc": [...], "msg": "... "type": "..."}]}
404	Запись не найдена	{"detail": "Expense not found"}
500	Внутренняя ошибка сервера	{"detail": "Internal server error"}
503	БД недоступна (нет соединения)	{"detail": "Database unavailable"}, эндпоинт /health возвращает 503

При недоступности PostgreSQL (например, контейнер упал) FastAPI перехватывает исключение `asyncpg.exceptions.ConnectionDoesNotExistError` и возвращает 503 `Service Unavailable`. Эндпоинт `GET /health` используется Kubernetes для liveness- и readiness-проб: при коде 200 под считается здоровым, при 503 - Kubernetes перезапускает под.

7. Целевая нагрузка

7.1. Параметры нагрузки

Система рассчитана на одного активного пользователя в рамках MVP. Тем не менее нагрузка спроектирована с запасом для корректной демонстрации масштабирования на этапе 4.3.

Параметр	Значение	Примечание
Количество пользователей	1	MVP
Запросов в день (штатно)	~10-20	Ручной ввод
Запросов в месяц	~300-600	Укладывается в free tier API Gateway
Пиковая нагрузка (нагруз. тест)	до 100 req/s	Для демонстрации HPA
Средний размер запроса	~1 КБ	JSON-объект расхода
Объём данных за год	<1 МБ	~1 000 записей
Время ответа (p95, штатно)	<200 мс	FastAPI + PostgreSQL

7.2. Нагрузочное тестирование (этап 4.3)

Для демонстрации HPA будет использован **Yandex.Tank** или **k6**. Сценарий: постепенное увеличение нагрузки с 1 до 100 req/s в течение 2 минут. При превышении 15% загрузки CPU Kubernetes автоматически создаёт новый под.

8. Масштабирование

8.1. Диаграмма масштабирования

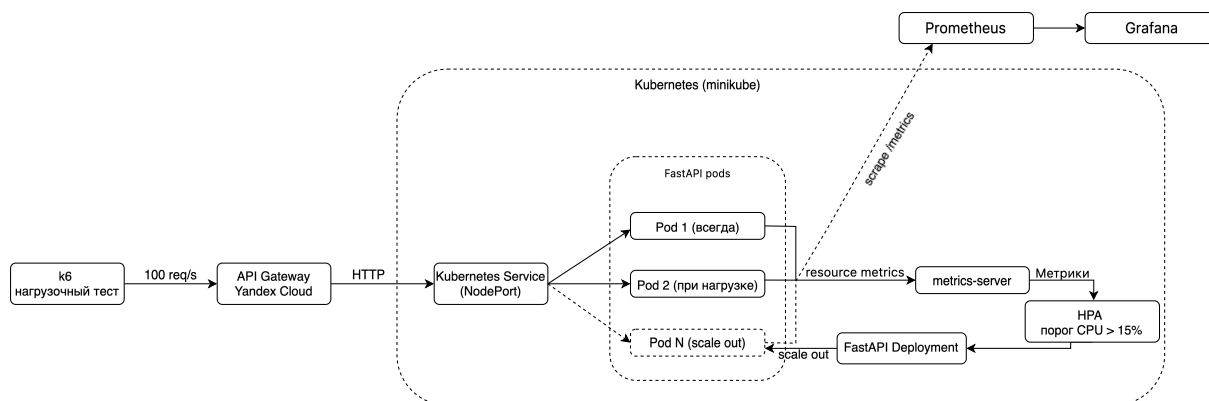


Рис. 4: Схема горизонтального масштабирования (HPA)

8.2. Масштабирование вверх (scale up)

- **Горизонтальное масштабирование приложения (HPA):** Kubernetes Horizontal Pod Autoscaler настроен на создание нового пода при превышении 15% загрузки CPU. Для корректной работы HPA в манифесте деплоя прописаны `resources.requests.cpu` и установлен `metrics-server`.
- **Вертикальное масштабирование ВМ:** При необходимости конфигурация Yandex Compute Cloud изменяется через Terraform - увеличиваются vCPU, RAM или размер диска без пересоздания инфраструктуры.
- **Переход на Managed PostgreSQL:** При росте нагрузки на БД PostgreSQL в Docker заменяется на Yandex Managed Service for PostgreSQL с репликацией.

8.3. Масштабирование вниз (scale down)

- **HPA scale down:** при снижении нагрузки Kubernetes автоматически уменьшает количество подов до минимума (1 под).
- **Остановка ВМ:** в период отсутствия разработки виртуальная машина останавливается через консоль Yandex Cloud или Terraform - плата за вычислительные ресурсы не взимается, оплачивается только хранилище.
- **FinOps:** описанная стратегия запуска компонентов только на период активной работы позволяет уложиться в грант 4 000 руб.

9. CI/CD и контейнеризация

9.1. Структура CI/CD пайплайна (этап 4.1, 4.4)

Этап	Название	Действие
1	test	Запуск pytest для бэкенда и Jest/аналога для фронтенда
2	build	Сборка Docker-образа
3	push	Публикация образа в Yandex Container Registry
4	deploy	CD: обновление деплоймента в Kubernetes (этап 4.4)

Инструмент: **GitHub Actions**. Триггер: push в ветку main.

9.2. Dockerfile

Многоэтапная сборка: на первом этапе устанавливаются зависимости, на втором - копируется только исполняемый код. Это уменьшает размер финального образа. Приложение запускается через `uvicorn` на порту 8000.

10. Мониторинг (этап 4.3)

Для мониторинга используется стек **Prometheus + Grafana**, развёрнутый в том же minikube-кластере.

- **Prometheus** будет собирать метрики с подов через эндпоинт `/metrics`, который добавляется в FastAPI на этапе 4.3 с помощью библиотеки `prometheus-fastapi-instrumentator`.
- **Grafana** отображает дашборды: количество запросов по эндпоинтам, время ответа (p50/p95), загрузка CPU/RAM по подам, количество активных подов (НРА).
- **metrics-server** установлен в кластере - обязательное условие для работы НРА по метрике CPU.